

domaci1_2022_0218

December 21, 2025

0.1 Deo 1

```
[ ]: import numpy as np
import skimage as ski
import matplotlib.pyplot as plt
import ipywidgets as widgets
from scipy import ndimage
from scipy.ndimage import label
```

```
[ ]: data_dir = "../sekvence/"
```

```
[ ]: def load_image(data_dir, path):
    """
    Funkcija koja služi
    za učitavanje slike
    """
    location = data_dir + path
    img = ski.util.img_as_float(ski.io.imread(location))
    return img
```

0.2 Detekcija i prefarbavanje plavih oblasti pomoću HSV kolor sistema

Da bismo izdvojili sve elemente na slici koji su **plave boje** i zatim ih obojili u neku drugu boju (**crvenu** u ovom slučaju), najpraktičnije je koristiti **HSV** kolor sistem.

0.2.1 Zašto HSV?

RGB kolor sistem nije pogodan za izolaciju određene boje, jer boja zavisi od kombinacije tri vrednosti (R, G i B), pa je teško definisati šta je plavo, crveno, itd.

HSV (Hue–Saturation–Value) je mnogo pogodniji:

- **H (Hue)** opisuje osnovnu boju i omogućava lako prepoznavanje da li piksel pripada plavoj boji.
- **S (Saturation)** predstavlja zasićenost boje, pa nam omogućava da izbegnemo bele, sive i blede tonove.
- **V (Value)** određuje osvetljenje, gde je **v=1** tamno za određenu boju a **v=0** svetlo za određenu boju.

Zahvaljujući ovom modelu, možemo vrlo jasno definisati masku koja će predstavljati plavu boju i odvojiti objekte sa tom bojom od ostatka slike.

0.2.2 Proces obrade

1. Ulaznu sliku konvertujemo iz **RGB u HSV**.
2. Pravimo **masku** na osnovu opsega vrednosti koje predstavljaju plavu boju (određeni opseg vrednosti Hue + filtriranje saturacije).
3. Koristimo masku da izdvojimo sve plave piksele.
4. Plave piksele prefarbamo u **crvenu boju**.
5. postavimo masku na nula tamo gde nam se ne nalazi odgovarajuća boja.
6. Rezultat kombinujemo sa ostatkom slike.
7. Dobijemo **crvene** autobuse umesto plavih

Ovaj pristup omogućava precizno izdvajanje plavih autobusa, uključujući samo one „življe“ tonove zahvaljujući saturaciji, dok se bleđunjav i sivkasti tonovi izostavljaju.

0.3 Određivanje opsega plave boje pomoću *widgets.interact()* funkcije

Plava boja se u HSV sistemu nalazi oko **240° Hue**.

Kada se Hue vrednosti skaliraju na opseg **0–1**, vrednost plave boje postaje:

$$H = \frac{240}{360} = 0.66$$

Da bismo precizno definisali koje vrednosti će ući u masku za plavu boju, koristimo **interaktivne slajdere** putem `widgets.interact()`.

Ovi slajderi omogućavaju vizuelno isprobavanje različitih opsega **Hue**, **Saturation**, kako bismo pronašli optimalne vrednosti koje pokrivaju sve plave autobuse na slici, a da pritom ne uključuju neželjene nijanse.

Nakon eksperimentisanja sa rasponima vrednosti pomoću interaktivnih kontrola, pronađeni su opsezi koji najbolje odgovaraju i oni su zatim prosleđeni funkcijama prilikom poziva.

Ovi opsezi nalaze se u kodu pri samom pozivu funkcija.

```
[ ]: def create_mask(img, lb_val, ub_val, sat_limit):  
    """Kreira masku u HSV color sistemu"""  
    hsv_image = ski.color.rgb2hsv(img)  
  
    #živahnije boje + opseg oko 0.66 za plavu  
    mask = ((hsv_image[:, :, 1] > sat_limit)  
            & ((hsv_image[:, :, 0] > lb_val) & (hsv_image[:, :, 0] < ub_val)))  
  
    img_copied = img.copy()  
    rgb_mask = np.stack([mask, mask, mask], -1)  
  
    #sve sto mi ne odgovara plavoj zakucanj na crno  
    img_copied[~rgb_mask] = 0
```

```
#crtalice
fig, ax = plt.subplots(nrows = 1, ncols = 2, figsize = (14, 8), dpi = 80)
ax[0].imshow(mask, cmap = 'gray')
ax[1].imshow(img_copied)
plt.tight_layout()
plt.show()
```

0.3.1 Opis funkcije `switch_blue_to_red`

Funkcija `switch_blue_to_red` menja boju plavih delova slike u crvenu boju u okviru HSV kolor sistema.

Ulazna HSV slika se najpre kopira kako bi se izbegla izmena originalnih podataka. Zatim se izdvaja **Hue (H)** komponenta slike. Na osnovu zadatih donje i gornje granice Hue vrednosti (`lb_val`, `ub_val`) i praga zasićenja (`sat_limit`), formira se maska koja identifikuje piksele koji pripadaju plavoj boji.

Za sve piksele koji zadovoljavaju ove uslove, Hue vrednost se postavlja na **0**, što u HSV prostoru odgovara crvenoj boji.

Nakon izmene Hue kanala, on se vraća u HSV sliku, koja se zatim konvertuje u RGB kolor sistem pomoću funkcije `hsv2rgb`. Na kraju, dobijena slika se prikazuje.

```
[ ]: def switch_blue_to_red(hsv_image, lb_val, ub_val, sat_limit):
    #copy hsv_image, copy H component
    img_hsv_out = hsv_image.copy()
    img_hsv_h = hsv_image[:, :, 0].copy()
    hsv_only_blue = ((hsv_image[:, :, 1] > sat_limit)
                     & ((hsv_image[:, :, 0] > lb_val) & (hsv_image[:, :, 0] < ub_val)))

    img_hsv_h[hsv_only_blue] = 0
    img_hsv_out[:, :, 0] = img_hsv_h
    rgb_img = ski.color.hsv2rgb(img_hsv_out)
    ski.io.imshow(rgb_img)
    plt.show()
```

0.3.2 Detekcija crvenih vozila i njihovo prefarbavanje korišćenjem HSV maski

Na slici se pojavljuju dve dominantne boje vozila: **plava** i **crvena**.

Nakon što smo izdvojili plava vozila pomoću maske za plavu nijansu, potrebno je kreirati i **masku za crvena vozila** kako bismo njima upravljali nezavisno.

U HSV modelu, crvena boja je specifična jer se nalazi **na početku i na kraju Hue kruga**:

- minimalna vrednost: **H = 0**
- maksimalna vrednost: **H = 1**

Zbog toga maska za crvenu boju obično obuhvata dva opsega Hue vrednosti (npr. nisko oko 0 i visoko oko 1), što omogućava da se obuhvate svi crveni tonovi na slici.

Prefarbavanje izdvojenih vozila Nakon što su oba vozila (crvena i plava) detektovana kroz svoje odgovarajuće maske, prosleđujemo ih funkciji koja vrši promenu boje.

Za prebojavanja je korišćena funkcija `switch_to_diff_color`, kojoj se zadaje nova Hue vrednost u HSV kolor sistemu.

Funkcija `switch_to_diff_color` radi na sličan način kao `switch_blue_to_red` samo što se koriste dve maske, maska za crvenu i maska za plavu.

Izbor nove boje Odabrana boja je **ljubičasta**, koja se u HSV prostoru nalazi približno na:

$$H \approx 0.84.$$

Ova vrednost se koristi kao boja u koju se farbaju oba vozila na slici.

Napomena o Saturation vrednosti Važno je istaći da se promena boje vrši sa:

$$S = 0.5$$

čime se dobija stabilna, zasićena nijansa bez previše bledih tonova.

Kombinacijom maski za plava i crvena vozila koje se izdvajaju u funkciji `switch_diff_color`, omogućeno je da se oba vozila vrlo lako prefarbaju u bilo koju ciljnu boju jednostavnim podešavanjem Hue vrednosti(`color_value`).

```
[ ]: def switch_to_diff_color(hsv_image, low_blue_val, upper_blue_val, sat_limit ,  
    ↪color_value):  
    #copy hsv_image, copy H component  
    img_hsv_out = hsv_image.copy()  
    img_hsv_h = hsv_image[:, :, 0].copy()  
  
    hsv_only_blue = ((hsv_image[:, :, 1] > sat_limit)  
        & ((hsv_image[:, :, 0] > low_blue_val) & (hsv_image[:, :, 0] < ↪  
    ↪upper_blue_val)))  
    hsv_only_red = ((hsv_image[:, :, 1] > sat_limit)  
        & ((hsv_image[:, :, 0] > 0.95) | (hsv_image[:, :, 0] < 0.05)))  
  
    img_hsv_h[hsv_only_blue] = color_value  
    img_hsv_h[hsv_only_red] = color_value  
  
    #img_hsv_h[img_hsv_h > 1] = img_hsv_h[img_hsv_h > 1] - 1  
    img_hsv_out[:, :, 0] = img_hsv_h  
    rgb_img = ski.color.hsv2rgb(img_hsv_out)  
    ski.io.imshow(rgb_img)  
    plt.show()
```

```
[ ]: #load images
gsp_1 = load_image(data_dir , "gsp1.jpg")
gsp_2 = load_image(data_dir , "gsp2.jpg")
gsp_3 = load_image(data_dir , "gsp3.jpg")

#make hsv images
hsv_gsp_1 = ski.color.rgb2hsv(gsp_1)
hsv_gsp_2 = ski.color.rgb2hsv(gsp_2)
hsv_gsp_3 = ski.color.rgb2hsv(gsp_3)

#make mask
create_mask(gsp_1, 0.6, 0.7, 0.3)
create_mask(gsp_2, 0.6, 0.78, 0.2)
create_mask(gsp_3, 0.6, 0.78, 0.25)

#switch colors
switch_blue_to_red(hsv_gsp_1, 0.6, 0.7, 0.15)
switch_blue_to_red(hsv_gsp_2, 0.6, 0.7, 0.1)
switch_blue_to_red(hsv_gsp_3, 0.6, 0.78, 0.25)
```

```
[ ]: # widgets.interact(create_mask, img = widgets.fixed(gsp_1), lb_val = (0, 1, 0.
    ↳01), ub_val = (0, 1, 0.01),
#           sat_limit = (0, 1, 0.01), l_v_limit = (0,1,0.01), u_v_limit
    ↳= (0, 1, 0.01))
# widgets.interact(create_mask, img = widgets.fixed(gsp_2), lb_val = (0, 1, 0.
    ↳02), ub_val = (0, 1, 0.01),
#           sat_limit = (0, 1, 0.01), l_v_limit = widgets.fixed(0),
    ↳u_v_limit = widgets.fixed(1))
# widgets.interact(create_mask, img = widgets.fixed(gsp_3), lb_val = (0, 1, 0.
    ↳02), ub_val = (0, 1, 0.02),
#           sat_limit = (0, 1, 0.01), l_v_limit = widgets.fixed(0),
    ↳u_v_limit = widgets.fixed(1))

# widgets.interact(switch_blue_to_red, hsv_image = widgets.fixed(hsv_gsp_1),
    ↳lb_val = (0, 1, 0.01), ub_val = (0, 1, 0.01),
#           sat_limit = (0, 1, 0.01))
# widgets.interact(switch_blue_to_red, hsv_image = widgets.fixed(hsv_gsp_2),
    ↳lb_val = (0, 1, 0.02), ub_val = (0, 1, 0.01),
#           sat_limit = (0, 1, 0.01))
# widgets.interact(switch_blue_to_red, hsv_image = widgets.fixed(hsv_gsp_3),
    ↳lb_val = (0, 1, 0.02), ub_val = (0, 1, 0.02),
#           sat_limit = (0, 1, 0.01))
```

```
[ ]: # Moja predlozena boja ljubicasta
```

```
#switch colors to purple
switch_to_diff_color(hsv_gsp_1, 0.6, 0.7, 0.15, 0.84)
switch_to_diff_color(hsv_gsp_2, 0.6, 0.70, 0.1, 0.84)
switch_to_diff_color(hsv_gsp_3, 0.6, 0.78, 0.25, 0.84)

#widgets.interact(switch_to_diff_color, hsv_image = widgets.fixed(hsv_gsp_1),
↳sat_limit = widgets.fixed(0.1),
        #low_blue_val = widgets.fixed(0.6), upper_blue_val = widgets.
↳fixed(0.7), color_value = (0, 1, 0.01))
```

0.4 Deo 1 Bonus

0.5 Izdvajanje autobusa sa slike – opis postupka

U ovom zadatku cilj mi je bio da izdvojim autobuse sa slike i da ih naknadno obradim (promenim boju), bez da utičem na ostale objekte u sceni kao što su ljudi, torbe ili pozadina.

Prvi korak je bio da sliku prebacim u HSV kolor sistem, jer je u tom prostoru mnogo lakše izdvojiti objekte na osnovu boje nego u RGB kolor sistemu. Nakon toga sam definisao opsege za plavu i crvenu boju, pošto autobusi na slici sadrže te boje.

Na osnovu tih opsega sam napravio masku koja označava sve piksele koji pripadaju plavim i crvenim delovima slike. Ova maska, međutim, ne sadrži samo autobuse, već i druge objekte slične boje.

Zatim sam nad tom maskom primenio označavanje povezanih komponenti, čime sam dobio sve objekte na slici kao zasebne celine. Svaki objekat sam posmatrao pojedinačno i za svaki sam sačuvao njegovu masku i veličinu izraženu kroz broj piksela.

Objekte sam potom sortirao po veličini, jer su autobusi u proseku znatno veći od ostalih objekata. Na taj način sam dobio listu objekata rangiranih od najvećih ka najmanjim.

Problem koji se pojavio je da jedan autobus često nije detektovan kao jedan objekat, već kao više manjih delova zbog prekida u boji, senki i refleksija. Zbog toga sam morao da pronađem način da te delove ponovo povežem.

Da bih to rešio, uveo sam filtriranje objekata po poziciji na slici (region of interest), jer se autobusi uvek nalaze u određenom delu slike. Tako sam mogao da izbacim objekte koji se nalaze van tog regiona i koji sigurno ne pripadaju autobusima.

Nakon toga sam grupisao objekte koji su prostorno blizu jedan drugom, to jest višak izbacio iz slike. Spajanjem njihovih maski dobio sam jednu zajedničku masku koja predstavlja ceo autobus.

Na kraju sam od svih izabranih objekata napravio jednu finalnu masku i primenio je na originalnu sliku. Ta maska mi je omogućila da menjam boju samo autobusa, dok su svi ostali delovi slike ostali nepromenjeni.

```
[ ]: def select_objects(rgb_img, lb_val, ub_val, sat_limit):
    """
    Izdvaja masku crvenih i plavih boja na slici
    objects = lista objekata {'mask':... , 'size':...}
```

```

    hsv_only_blue = maska plave, hsv_only_red = crvena maska,
    mask = crvena + plava maska
    vraća listu objekata na slici obuhvaćenih maskom
    """
    hsv_image = ski.color.rgb2hsv(rgb_img)

    hsv_only_blue = ((hsv_image[:, :, 1] > sat_limit)
                     & ((hsv_image[:, :, 0] > lb_val) & (hsv_image[:, :, 0] < ub_val)))
    hsv_only_red = ((hsv_image[:, :, 1] > sat_limit)
                   & ((hsv_image[:, :, 0] > 0.95) | (hsv_image[:, :, 0] < 0.05)))

    mask = hsv_only_red | hsv_only_blue

    labeled, num_objects = label(mask)
    objects = []

    for obj_id in range(1, num_objects + 1):
        obj_mask = (labeled == obj_id)
        obj_size = np.sum(obj_mask) # broj piksela

        objects.append({
            'mask': obj_mask,
            'size': obj_size
        })

    # sortiramo po veličini opadajuće
    objects.sort(key=lambda x: x['size'], reverse=True)

    return objects

```

```

[ ]: def catch_obj_indices(objects, y_range, x_range):
    """
    objects: lista objekata {'mask': ..., 'size': ...}
    y_range: tuple (min_y, max_y)
    x_range: tuple (min_x, max_x)

    Vraća np.array indeksa objekata koji imaju bar deo maske unutar zadatog regiona.
    """
    min_y, max_y = y_range
    min_x, max_x = x_range

    # using list for indices and at the end just convert it to the np.array
    indices = [i for i, obj in enumerate(objects) if np.any(obj['mask'][min_y:
    max_y, min_x:max_x])]

```

```
return np.array(indices, dtype=int)
```

```
[ ]: def merge_objects_by_indices(objects, indices, min_size = 50, skip_value = None):
    """
    objects: lista objekata {'mask': ..., 'size': ...}
    indices: np.array indeksa objekata koje želimo spojiti
    min_size: minimalna veličina da se uključi u finalnu masku
    skip_value: neki veći opseg koji je u masci ali ne želimo da nam bude na_
    ↪slici
    (npr. krov kuće na prvoj slici)
    """
    merged_mask = np.zeros_like(objects[0]['mask'], dtype=bool)
    for i in indices:
        if objects[i]['size'] >= min_size and i not in skip_value:
            merged_mask |= objects[i]['mask']
    return merged_mask
```

```
[ ]: def switch_colors(img, objects, mask, color_value):
    """Menja boju piksela označenih maskom
    promenom Hue vrednosti u HSV kolor sistemu."""
    hsv_image = ski.color.rgb2hsv(img)
    img_hsv_out = hsv_image.copy()
    img_hsv_h = hsv_image[:, :, 0].copy()

    img_hsv_h[mask] = color_value
    img_hsv_out[:, :, 0] = img_hsv_h
    rgb_img = ski.color.hsv2rgb(img_hsv_out)
    ski.io.imshow(rgb_img)
    plt.show()
```

```
[ ]: def which_indices_to_skip(gsp_img, gsp_objects, min_size = 300):
    """Stampa slike sa delovima maske kako bi mogli uociti sta da
    se izbací iz konacne maske """

    for idx, obj in enumerate(gsp_objects):
        if obj['size'] <= min_size:
            continue

    fig, ax = plt.subplots(1, 2, dpi=80, figsize=(12, 8))
    ax[0].imshow(gsp_img)
    ax[1].imshow(obj['mask'], cmap='gray')
    ax[1].set_title(f'Object index: {idx}, size: {obj["size"]}')
    plt.show()
```

0.5.1 Funkcija which_indices_to_stop

Ova funkcija služi za vizuelni pregled najvećih detektovanih objekata na slici.

Iako je neefikasna (prikazuje prvih ~20 najvećih objekata), omogućila je da se ručno identifikuju objekti koji smetaju (npr. ljudi, torbe, jakne). Na osnovu tog pregleda formirane su `skip_value` liste koje se kasnije koriste za izbacivanje neželjenih objekata iz obrade.

```
[ ]: #load images
gsp_1 = load_image(data_dir , "gsp1.jpg")
gsp_2 = load_image(data_dir , "gsp2.jpg")
gsp_3 = load_image(data_dir , "gsp3.jpg")

# shows which indices are the problem by showing every object with area larger
↳ than min_area
gsp_objects_3 = select_objects(gsp_3, lb_val=0.6, ub_val=0.78, sat_limit=0.25)
which_indices_to_skip(gsp_3, gsp_objects_3)
```

```
[ ]: gsp_1 = load_image(data_dir , "gsp1.jpg")
gsp_2 = load_image(data_dir , "gsp2.jpg")
gsp_3 = load_image(data_dir , "gsp3.jpg")

# store every object possible with certain color (blue, red)
gsp_objects_1 = select_objects(gsp_1, lb_val=0.6, ub_val=0.7, sat_limit=0.15)
gsp_objects_3 = select_objects(gsp_3, lb_val=0.6, ub_val=0.78, sat_limit=0.25)

# return array of object indices
gsp_ranged_obj1 = catch_obj_indices(gsp_objects_1, y_range = (300, 1100),
↳ x_range = (0, 2500))
gsp_ranged_obj3 = catch_obj_indices(gsp_objects_3, y_range = (400, 750),
↳ x_range = (100, 1100))

# which objects do i need to skip?
skip_value_1 = np.array([3])
skip_value_3 = np.array([3, 6, 7, 9, 16, 21, 22, 36])

# function that merges every important parts of an image in a mask
merged_mask1 = merge_objects_by_indices(gsp_objects_1, gsp_ranged_obj1,
↳ min_size= 300, skip_value = skip_value_1)
merged_mask3 = merge_objects_by_indices(gsp_objects_3, gsp_ranged_obj3,
↳ min_size=50, skip_value = skip_value_3)

# now that everything is merged we can color busses to determined color
# color in which i will paint busses is orange in this case
switch_colors(gsp_1, [{'mask': merged_mask1, 'size': np.sum(merged_mask1)}],
↳ merged_mask1, color_value = 0.08)
switch_colors(gsp_3, [{'mask': merged_mask3, 'size': np.sum(merged_mask3)}],
↳ merged_mask3, color_value = 0.08)
```

0.6 Deo 2

```
[ ]: import imageio
      imageio.plugins.freeimage.download()
```

a)

0.6.1 calculate_value

Funkcija **calculate_value** određuje vrednosti skalarnih faktora koje je potrebno odabrati da bi se dobile željene slike pod tačkom **a**:

1. Pomoću funkcije `.copy()` kopira se vrednost slika i dobijamo 3 instance slika `img1`, `img2`, `img3`.
2. Potom se pomoću parametara slika linearno skalira tako što se slike množe odgovarajućim parametrima.
3. Pomoću funkcije `np.clip` clippuje se na vrednost 1 kao maksimalnu vrednost slika i 0 kao minimalnu.
Dobijamo tri instance slika `img_1`, `img_2`, `img_3`.
4. Na kraju svaki element slike se stepenjuje faktorom, za sve slike:

$$\gamma = \frac{1}{2.2}$$

5. Nakon stepenovanja sa faktorom gama, slike se prikazuju na jednoj figuri radi lakšeg upoređivanja rezultata.
6. Pomoću funkcije `widgets.interact()` menjaju se vrednosti dok se ne dobiju slike koje odgovaraju onima zadatim u okviru zadatka.
7. Nakon nekoliko pokušaja dolazi se do zaključka da su najbliže vrednosti:

A = 1.60, B = 15.20, C = 50.00

Napomena:

Da su bile zadate referentne slike, zadatak bi se mogao rešiti tako što bi se one učitale, a zatim bi se računala razlika između generisanih slika i referentnih slika. Kada bi dobijena razlika bila bliska nuli, moglo bi se zaključiti da su pronađene odgovarajuće vrednosti parametara.

```
[ ]: img_path = data_dir + "sea.hdr"
      img = imageio.v2.imread(img_path, format="HDR-FI")
      img = img / img.max()
      gamma = 1 / 2.2
```

```
[ ]: def calculate_value(A, B, C, gamma, img):
      img_1 = img.copy()
      img_2 = img.copy()
      img_3 = img.copy()
```

```

img_1 *= A
img_2 *= B
img_3 *= C

img_1 = np.clip(img_1, a_min = 0, a_max = 1)
img_1 **= gamma
img_2 = np.clip(img_2, a_min = 0, a_max = 1)
img_2 **= gamma
img_3 = np.clip(img_3, a_min = 0, a_max = 1)
img_3 **= gamma

fig, ax = plt.subplots(nrows = 1, ncols = 3, dpi = 80, figsize = (14,8))
ax[0].imshow(img_1); ax[0].set_title("A = {:.4f}".format(A))
ax[1].imshow(img_2); ax[1].set_title("B = {:.4f}".format(B))
ax[2].imshow(img_3); ax[2].set_title("C = {:.4f}".format(C))
plt.tight_layout()
plt.show()
print(img.max())

```

```

[ ]: # widgets.interact(calculate_value, A = (0, 100, 0.2), B = (0, 100, 0.2) , C =
    ↪(0, 100, 0.2),
#                                     gamma = widgets.fixed(gamma), img =
    ↪widgets.fixed(img))

```

b)

```

[ ]: img_path = data_dir + "sea.hdr"
img = imageio.v2.imread(img_path, format="HDR-FI")

```

```

[ ]: def log_transform(f, c):
    f /= f.max()
    g = np.log(1 + (10**c)*f)/np.log(1+10**c)
    # show img
    fig, axis = plt.subplots(dpi = 80, figsize = (12, 8))
    axis.set_title("Logaritamsko mapiranje")
    plt.imshow(g)

```

```

[ ]: def power_function(f, c = 0.5):
    f /= f.max()
    g = f**c
    # show img
    fig, axis = plt.subplots(dpi = 80, figsize = (12, 8))
    axis.set_title("Stepeno mapiranje:")
    ski.io.imshow(g)

```

- **Logaritamska funkcija:**

- Za parametar **c = 3.8** dobijaju se najbolji rezultati.
- Generalno, vrednosti **u opsegu 3.5 – 5** daju dobre rezultate, dok nakon toga kvalitet počinje da opada.
- Za **c > 5**, slika postaje svetlija kako se **c** povećava, odnosno slika ima manji kontrast.
- Za **c < 5**, slika postaje tamnija, odnosno kontrast slik se povećava.

```
[ ]: #widgets.interact(log_transform, f = widgets.fixed(img), c=(0,10,0.2))
```

- **Stepena funkcija:**

- Kada se vrednost **c povećava**, slika postaje tamnija, kontrast slike se povećava.
- Kada se vrednost **c smanjuje**, slika postaje svetlija i čitljivija, takodje se i kontrast slike smanjuje.
- Za **c = 0.3** ili **c = 0.2** dobijaju se najbolji rezultati kod stepene funkcije.
- Za **c > 1.3**, većina detalja nestaje i jedino se sunce može videti
- Interesantno je zapaziti da nakon određene vrednosti sunce postaje zeleno.

```
[ ]: #widgets.interact(power_function, f = widgets.fixed(img), c=(0,10,0.1))
```

c)

```
[ ]: log_transform_img = log_transform(img, 3.8)
power_func_img = power_function(img, 0.3)
```

Moguće je koristiti i sledeće tehnike:

1. **Contrast Stretching (rescale_intensity)**

- Proširuje opseg intenziteta piksela kako bi tamni delovi postali tamniji, a svetli svetliji.
- Može se koristiti na HDR slikama da se ceo dinamički opseg bolje prikaže na običnom ekranu.

2. **Thresholding**

- Pretvara sliku u binarnu (crno/belo) tako što svi pikseli iznad određene vrednosti postaju 1, a ispod 0.
- Koristi se za izdvajanje objekata ili segmentaciju određenih delova scene.

3. **Bit-Plane Slicing**

Analizira i prikazuje pojedinačne bitove svakog piksela. Najznačajniji bitovi daju osnovni oblik, dok niži daju fine detalje. Može se koristiti za vizualizaciju detalja u različitim opsezima intenziteta.

4. **Clipping / Saturation**

- Sve vrednosti koje prelaze određeni prag se “seku” na maksimalnu ili minimalnu vrednost.
- Može se koristiti za kontrolu preosvetljenih ili potamnelih delova pri linearnom skaliranju HDR slike.

5. CLAHE (Contrast Limited Adaptive Histogram Equalization)

- Lokalno prilagođava kontrast po blokovima slike, ograničavajući prekomernu saturaciju da ne bi došlo do šuma.
- Idealan za HDR slike kod kojih su histogrami po različiti za različite delove slike. U tom slučaju pojačavamo lokalne detalje bez da uništi globalni kontrast. — U kodu ispod korišćena je tehnika adaptivne ekvivalizacije histograma:

```
[ ]: img_path = data_dir + "sea.hdr"
img = imageio.v2.imread(img_path, format="HDR-FI")
img_float = ski.util.img_as_float(img)
img_float = img_float / np.max(img_float)
height, width = img.shape[:2]
img_out = ski.exposure.equalize_adapthist(img_float, (height//8, width//8),
    ↪ clip_limit=0.2, nbins=256)
ski.io.imshow(img_out)
plt.show()
```

0.6.2 Efekat adaptivne ekvivalizacije histograma (CLAHE) na HDR sliku

U primeru smo primenili **adaptivnu ekvivalizaciju histograma** na HDR sliku (`sea.hdr`).

Da se Primetiti nekoliko stvari:

1. **Nebo se razmazuje** – ovo je očekivano jer smo postavili **clip limit = 0.2**.
2. **Zašto dolazi do razmazivanja?**
 - CLAHE funkcija ograničava maksimalni nagib lokalnog histograma prema `clip_limit`.
 - Pikseli sa ekstremnim vrednostima (kao što je svetlo nebo u HDR slici) su „**zakucani**“ na **maksimalne vrednosti**.
 - Zbog toga lokalna kontrastna korekcija pravi efekat izravnavanja svetlih delova, pa nebo izgleda razmazano.
3. **Opšti efekat**
 - CLAHE poboljšava kontrast u tamnijim i srednjim tonovima.
 - Svetli tonovi, posebno oni iznad clip limita, se „preskaču“ ili maksimalno naglašavaju, što može dovesti do gubitka detalja u vrlo svetlim delovima.

-
- Postavljanje `clip_limit`-a je ključno za kontrolu kontrasta.

- Na HDR slikama sa velikim dinamičkim opsegom, svetli delovi će se brzo „zakucati“ ako je clip limit previsok, što stvara efekat razmazivanja.

0.7 Deo 3

Funkcija izoštrava sliku tako što prvo konvertuje RGB sliku u YUV kolor sistem i izdvoji **Y kanal** (osvetljenost).

Zatim se Y kanal zamagli pomoću **niskopropusne maske** (5x5 matrica je uzeta moglo je i druge veličine), čime dobijamo zamućenu verziju slike. Bitno je da se подели sa zbirom u matrici jer je važno da nam na kraju rezultat bude 1 u maski.

Detalji slike se izdvajaju kao razlika između originalnog Y kanala i zamućene slike (Y - zamućeno). Na originalni Y kanal se dodaje ovaj „višak detalja“ pomnožen sa faktorom izoštravanja (**sharpen_factor**), čime se postiže efekat izoštravanja.

Posle se modifikovani Y kanal kombinuje nazad sa U i V kanalima, i konvertuje u RGB prostor.

Na kraju se vrednosti piksela koje su izašle iz opsega [0,1] ograniče tako da budu u validnom opsegu [0,1].

Nakon ovoga slika se prebaci u rgb domen i prikaže klasično pomoću plt.show() funkcije. Na slici se prikazuju originalna i izoštrana slika za vizuelno poređenje.

```
[ ]: def sharpen_image(img_rgb, sharpen_factor):
    img_yuv= ski.color.rgb2yuv(img_rgb)
    img_y = img_yuv[:, :, 0]
    low_pass_mask = np.ones(shape = (5,5)) / 25

    # calculate blurred part of the image
    img_y_blurred = ndimage.correlate(img_y, low_pass_mask)

    # details = original_image - blurred_image
    img_y_details = img_y - img_y_blurred

    # we get enchanced version of an image as enchanced_image = image_details +
    ↪ original_image
    img_y_sharp = img_y + sharpen_factor * img_y_details

    img_yuv_out = np.zeros(np.shape(img_yuv))
    img_yuv_out[:, :, 0] = img_y_sharp
    img_yuv_out[:, :, 1:3] = img_yuv[:, :, 1:3]

    # back to rgb, and clip image if some value has gotten out of the range (0,
    ↪ 1)
    img_rgb_out = ski.color.yuv2rgb(img_yuv_out)
    img_rgb_out = np.clip(img_rgb_out, 0, 1)

    # plot image
    fig, axes = plt.subplots(nrows = 1, ncols = 2, figsize = (16, 8), dpi = 80)
    ax = axes.ravel()
```

```

ax[0].imshow(img_rgb, vmin=0, vmax=1);
ax[0].set_title('Ulazna slika:', fontsize=16)
ax[1].imshow(img_rgb_out, vmin=0, vmax=1);
ax[1].set_title('Izoštrena slika:', fontsize=16)

plt.tight_layout()
plt.show()
return img_rgb_out

```

```

[ ]: img_rgb = load_image(data_dir, "miner.jpg")
img_rgb_out = sharpen_image(img_rgb, 2)
ski.io.imsave("miner_sharp.jpg", ski.img_as_ubyte(img_rgb_out))

```

Za vrednosti **sharpen_factor** > 2 vidi se izoštravanje slike.

Međutim trebalo bi reći da za vrednosti **sharpen_factor** > 4 slika postaje neprirodna,

Pa samim tim slika najbolje izgleda za vrednosti: **2 ≤ sharpen_factor ≤ 4**.

U ovom slučaju izabrao sam da je **sharpen_factor** = 2 kako bi se precizno video efekat izoštravanja slike.

Kako se **sharpen_factor** povećava: - visoke frekvencije u slici bivaju sve više pojačane, - šum postaje izraženiji, - ivice počinju da deluju neprirodno, - javljaju se artefakti poput halo-efekta oko ivica.

Zbog toga slika za veće vrednosti ovog parametra gubi prirodan izgled i postaje vizuelno neprijatna.

```

[ ]: #widgets.interact(sharpen_image, img_rgb = widgets.fixed(img_rgb),
    ↪sharpen_factor = (0, 10, 0.1))

```

0.8 DEO 4

```

[ ]: import time

```

0.9 CLAHE algoritam – opis postupka

Funkcija **dosCLAHE** predstavlja ručnu implementaciju algoritma **za adaptivnu ekvivalizaciju histograma** bez korišćenja ugrađenih funkcija za histogram, ekvalizaciju i interpolaciju.

Ulazna slika može biti: - **2D matrica** – grayscale slika

- **3D matrica** – RGB slika (uint8 format)

Izlazna slika ima **iste dimenzije kao ulazna**.

0.10 1. Validacija parametara

Na početku se proverava: - da li je slika tipa uint8 - da li je parametar limit tipa int ili float - da li je numTiles dvodimenzioni niz (broj blokova po visini i širini)

Ako parametri nisu prosleđeni, koriste se podrazumevane vrednosti: - numTiles = (8, 8) - limit = 0.01

```
[ ]: def validateParameters(imgIn, numTiles, limit):
    if numTiles is None:
        numTiles = [8, 8]
    if limit is None:
        limit = 0.01

    if imgIn.dtype != np.uint8:
        raise TypeError("Image must be of type uint8")
    if not isinstance(limit, (float,int)):
        raise TypeError("limit must be float or int")
    if not isinstance(numTiles,(list,tuple)) or len(numTiles)!=2:
        raise TypeError("numTiles must be list/tuple of length 2")
    return numTiles, limit
```

0.11 2. Proširivanje slike (padding)

Ako dimenzije slike **nisu deljive** sa brojem blokova: - slika se proširuje ponavljanjem **poslednjeg reda i/ili kolone** - proširivanje se vrši samo onoliko puta koliko je potrebno

Na ovaj način se obezbeđuje da svi blokovi imaju istu veličinu.

```
[ ]: def padImage(img, pad_h, pad_w):
    if pad_h>0:
        last_row = img[-1:,...]
        img = np.vstack([img, np.repeat(last_row, pad_h, axis=0)])
    if pad_w>0:
        last_col = img[:, -1:,...]
        img = np.hstack([img, np.repeat(last_col, pad_w, axis=1)])
    return img
```

```
[ ]: def fulfillImage(imgIn, numTiles):
    h_tiles, v_tiles = numTiles
    h,w = imgIn.shape[:2]
    new_h, new_w = h, w
    while new_h % h_tiles !=0:
        new_h +=1
    while new_w % v_tiles !=0:
        new_w +=1
    return padImage(imgIn.copy(), new_h-h, new_w-w)
```

0.12 3. Podela slike na blokove

Proširena slika se deli na: - $h_blocks \times w_blocks$ blokova - svaki blok ima dimenzije $block_h \times block_w$

Blokovi se čuvaju u posebnoj 4D ili 5D matrici (u zavisnosti da li je slika grayscale ili RGB).

```
[ ]: def createBlocks(img, h_blocks, v_blocks):
    """
    Deli sliku na blokove jednake veličine.

    Parametri:
    img : Ulazna slika (grayscale ili RGB).
    h_blocks : Broj blokova po visini.
    v_blocks : Broj blokova po širini.

    Povratna vrednost:
    blocks :
        Niz blokova dimenzija:
        - grayscale: (h_blocks, v_blocks, block_h, block_w)
        - RGB: (h_blocks, v_blocks, block_h, block_w, channels)
    """
    h, w = img.shape[:2]
    block_h, block_w = h // h_blocks, w // v_blocks

    if img.ndim == 2: # grayscale
        blocks = np.zeros((h_blocks, v_blocks, block_h, block_w),
                          dtype=img.dtype)
    else: # RGB
        channels = img.shape[2]
        blocks = np.zeros((h_blocks, v_blocks, block_h, block_w, channels),
                          dtype=img.dtype)

    for i in range(h_blocks):
        for j in range(v_blocks):
            y0, y1 = i * block_h, (i + 1) * block_h
            x0, x1 = j * block_w, (j + 1) * block_w

            if img.ndim == 2:
                blocks[i, j] = img[y0:y1, x0:x1]
            else:
                blocks[i, j] = img[y0:y1, x0:x1, :]

    return blocks
```

0.13 4. Računanje histograma po blokovima

Za svaki blok: - ručno se računa histogram sa 256 binova - Objasniću kako računam histogram na primeru:

- recimo da su vrednosti 5,10,5,1,1 vrednosti niza će biti: `hist[1] = 2` jer se vrednost 1 pojavila dva puta, `hist[5] = 2`, `hist[10] = 1`, `hist[ostatak] = 0` - histogram se **clip-uje** pomoću parametra `limit`
- višak vrednosti se ravnomerno redistribuira

```
[ ]: def createHistogram(block, limit):
    # grayscale
    if block.ndim == 2:
        hist = [0]*256
        for val in (block*255).astype(int).flatten():
            hist[val] +=1
        return clipHistogram(hist, limit, block.size)
    else:
        # V = [0, 1]->[0, 255]-> change into array with data of type integer
        # histogram is calculated only for V channel of the HSV color system
        block_value = (block[..., 2]*255).astype(int)
        hist = [0]*256
        for val in block_value.flatten():
            hist[val] += 1
        return clipHistogram(hist, limit, block.size)
```

```
[ ]: def clipHistogram(hist, limit, num_pixels):
    clip_limit = int(num_pixels*limit)
    excess = [h-clip_limit if h>clip_limit else 0 for h in hist]
    hist = [h if h<=clip_limit else clip_limit for h in hist]
    redistribute = sum(excess)//256
    hist = [h+redistribute for h in hist]
    return hist
```

```
[ ]: def cumulate(h, pixels_per_block):
    cdf = []
    s = 0
    for val in h:
        s += val
        v = s / pixels_per_block
        if v>1:
            v=1
        cdf.append(v)
    return cdf
```

0.14 5. Računanje transformacione funkcije T (CDF)

Iz clip-ovanog histograma računa se kumulativna funkcija raspodele (CDF):

$$T(i) = \frac{\sum_{k=0}^i h(k)}{N}$$

gde je: - $h(k)$ vrednost histograma - N broj piksela u bloku

funkcija `clip_arr` je funkcija koja simulira `np.clip()` funkciju i zakucava vrednosti na gornji, odnosno donji opseg u slucaju da su vrednosti iznad opsega.

```
[ ]: def calculateT(hist, pixels_per_block):
    # if isinstance(hist, list) and isinstance(hist[0], list):
    #     # RGB
    #     return [cumulate(h, pixels_per_block) for h in hist]
    # else:
    return cumulate(hist, pixels_per_block)
```

```
[ ]: def clip_arr(arr, min_val=0, max_val=1):
    arr[arr < min_val] = min_val
    arr[arr > max_val] = max_val
    return arr
```

0.15 6. Bilinearna interpolacija između blokova

Za svaki blok koristi se do četiri susedne transformacione funkcije: - T1 – trenutni blok - T2 – blok desno - T3 – blok ispod - T4 – dijagonalni blok

Ako neki susedni blok ne postoji (ivice slike): - pretpostavlja se da je njegova transformacija jednaka T1 - time mu se dodeljuje težina 0, odnosno kao da blok ne učestvuje u bilinearnoj transformaciji između 4 bloka

Za svaki piksel u bloku računaju se relativne koordinate: - $dx = x / (\text{širina_bloka} - 1) = b - a$ - $dy = y / (\text{visina_bloka} - 1) = d - c$ - $a = 1 - b - c = 1 - d$

Zatim se vrši bilinearna interpolacija:

$$sH1 = \frac{a \cdot T_1 + b \cdot T_2}{a + b}$$

$$sH2 = \frac{a \cdot T_3 + b \cdot T_4}{a + b}$$

Na kraju izlazni piksel računamo kao:

$$s = \frac{c \cdot sH1 + d \cdot sH2}{c + d}$$

Ovaj način bilinearne transformacije uzet je sa predavanja sa predmeta pa je odatle i implementirana ova funkcionalnost.

```
[ ]: def bilinearInterpolation(T1, T2, T3, T4, block):
    block_h, block_w = block.shape[:2]

    # coordinates of every pixel in a block
    ys, xs = np.meshgrid(np.arange(block_h), np.arange(block_w), indexing='ij')

    # dx = x pixel coordinate / block width
    dx = xs / (block_w - 1) if block_w > 1 else 0
```

```

# dy = y pixel coordinate / block heigth
dy = ys / (block_h - 1) if block_h > 1 else 0

a = 1 - dx
b = dx
c = 1 - dy
d = dy

if block.ndim == 2: # grayscale

    # convert u int indices
    vals = (block * 255).astype(int)

    # make T in np.arrays if they aren't np.arrays already
    T1_arr = np.asarray(T1)
    T2_arr = np.asarray(T2)
    T3_arr = np.asarray(T3)
    T4_arr = np.asarray(T4)

    # bilinear interpolation
    sH1 = a * T1_arr[vals] + b * T2_arr[vals]
    sH2 = a * T3_arr[vals] + b * T4_arr[vals]
    sV = c * sH1 + d * sH2

    # scale back to the range of (0, 255) and clip
    sV_clip = clip_arr(sV*255, 0, 255)
    result = sV_clip.astype(np.uint8)

    #return me block through whom clahe was executed
    return result
else: # HSV format - only V channel should be transformed
    result = block.copy()

    # Using only V channel of HSV for transformacion
    V_channel = block[..., 2] # V channel in in range 0-1
    vals = (V_channel * 255).astype(int) # indices from 0-255

    # change into T np.arrays
    T1_arr = np.asarray(T1)
    T2_arr = np.asarray(T2)
    T3_arr = np.asarray(T3)
    T4_arr = np.asarray(T4)

    # bilinear interpolation in v channel
    sH1 = a * T1_arr[vals] + b * T2_arr[vals]
    sH2 = a * T3_arr[vals] + b * T4_arr[vals]

```

```

sV = c * sH1 + d * sH2

# update V channel (H and S are left unchanged)
result[..., 2] = clip_arr(sV, 0, 1)

# return me block through whom clahe was executed
return result

```

0.16 7. Ograničavanje vrednosti piksela

Dobijene vrednosti se: - ograničavaju na opseg [0, 255] za grayscale sliku, odnosno na (0, 1) opseg za RGB sliku. - konvertuju nazad u tip uint8 za grayscale.

0.17 8. Formiranje izlazne slike

Interpolirani blokovi se upisuju u izlaznu sliku. Na kraju se uklanja padding i izlazna slika dobija iste dimenzije kao ulazna.

```

[ ]: def interpolate(imgIn, h_tiles, w_tiles, blocks, limit, pixels_per_block,
    padded_img, tile_h, tile_w, imgOut):
    for i in range(h_tiles):
        for j in range(w_tiles):
            # make histogram for currrent block
            hist_current = createHistogram(blocks[i, j], limit)
            T1 = calculateT(hist_current, pixels_per_block)

            # T2 - block to the right side of T1 (if exists)
            # if i am in the last column T2 = T1 just compute T2 by formula
            # similar process of thinking for T3 and T4
            if j == w_tiles - 1:
                T2 = T1
            else:
                hist_right = createHistogram(blocks[i, j + 1], limit)
                T2 = calculateT(hist_right, pixels_per_block)

            # T3 - block below the T1 block if T3 exists
            if i == h_tiles - 1:
                T3 = T1
            else:
                hist_bottom = createHistogram(blocks[i + 1, j], limit)
                T3 = calculateT(hist_bottom, pixels_per_block)

            # T4 - block which is diagonally right from T1
            if i == h_tiles - 1 or j == w_tiles - 1:
                T4 = T1
            else:

```

```

        hist_bottom_right = createHistogram(blocks[i + 1, j + 1], limit)
        T4 = calculateT(hist_bottom_right, pixels_per_block)

        # We take original block from an image
        block = padded_img[i * tile_h:(i + 1) * tile_h, j * tile_w:(j + 1) *
↪ tile_w]

        # we pull that block through bilinear transformation
        interpolated_block = bilinearInterpolation(T1, T2, T3, T4, block)

        # add computed block to the output image
        imgOut[i * tile_h:(i + 1) * tile_h, j * tile_w:(j + 1) * tile_w] =
↪ interpolated_block

        # return part of the image which correspond to the original image (without
↪ padding-a)
        return imgOut[:imgIn.shape[0], :imgIn.shape[1]]

```

0.18 9. Realizacija funkcije dosCLAHE

Na kraju dolazimo do glavne funkcije u okviru implementacije CLAHE algoritma – **dosCLAHE**. Ova funkcija objedinjuje sve prethodno definisane korake i predstavlja centralnu tačku obrade slike.

Redosled operacija unutar funkcije je sledeći:

1. Provera ulaznih parametara

Na samom početku proverava se ispravnost prosleđenih parametara (tip slike, broj blokova `numTiles`, vrednost `clip_limit`).

Ovim se obezbeđuje da dalja obrada bude stabilna i konzistentna.

2. Dopunjavanje (padding) slike

Ulazna slika se po potrebi dopunjava tako da njene dimenzije budu deljive brojem blokova.

Time se omogućava pravilna podela slike na jednake blokove.

3. Konverzija u HSV kolor sistem (ako je slika RGB)

Ukoliko je ulazna slika u RGB formatu, vrši se konverzija u HSV.

Obrada se zatim vrši isključivo nad **V (Value)** kanalom, dok **H** i **S** kanali ostaju nepromenjeni, čime se očuvaju informacije o boji.

4. Interpolacija i lokalna ekvalizacija kontrasta

Poziva se funkcija koja vrši:

- podelu slike na blokove
- računanje lokalnih histograma
- clipping histograma
- računanje transformacionih funkcija

- **bilinearnu interpolaciju** između četiri susedna bloka

Kao rezultat ovog koraka dobija se izmenjena slika sa poboljšanim lokalnim kontrastom.

5. Vraćanje u originalni kolor sistem i prikaz rezultata

Nakon obrade, slika se:

- vraća nazad u RGB format (ukoliko je inicijalno bila RGB)
- skalira u odgovarajući opseg i tip podataka

Dobijena slika se zatim koristi u fazi testiranja i vizuelno upoređuje sa originalom.

```
[ ]: def rescale_intensity(padded_img, hsv_padded_img, imgIn, h_tiles, w_tiles,
    ↪ blocks, limit, pixels_per_block, tile_h, tile_w):
    # make output with the same dimensions as input
    if padded_img.ndim == 3: # RGB => HSV
        imgOut = np.zeros_like(hsv_padded_img)
    else: # Grayscale
        imgOut = np.zeros_like(hsv_padded_img)

    output_img = interpolate(imgIn, h_tiles, w_tiles, blocks, limit,
    ↪ pixels_per_block, hsv_padded_img, tile_h, tile_w,
        imgOut)

    # converting back to the corresponding format
    if padded_img.ndim == 3: # image was RGB , we convert it back from HSV
        output_img_rgb = ski.color.hsv2rgb(output_img)

        # restrict to the valid values and scale them to the (0, 255) range
        output_img_rgb = clip_arr(output_img_rgb, 0, 1)
        return (output_img_rgb * 255).astype(np.uint8)
    else: # Grayscale - turn it back to the uint8
        output_gray = clip_arr(output_img, 0, 255).astype(np.uint8)
        return output_gray
```

```
[ ]: def dosCLAHE(imgIn, numTiles=None, limit=None):
    numTiles, limit = validateParameters(imgIn, numTiles, limit)
    padded_img = fulfillImage(imgIn, numTiles)

    # is image rgb or grayscale?
    if padded_img.ndim == 3: # RGB
        # convert in HSV
        hsv_padded_img = ski.color.rgb2hsv(padded_img)
    else: # Grayscale
        # normalize grayscale to the (0,1) range
        hsv_padded_img = padded_img.astype(np.float32) / 255.0

    h_tiles, w_tiles = numTiles
```

```

h_img, w_img = hsv_padded_img.shape[:2]
tile_h, tile_w = h_img // h_tiles, w_img // w_tiles

pixels_per_block = tile_h * tile_w
blocks = createBlocks(hsv_padded_img, h_tiles, w_tiles)
out_img = rescale_intensity(padded_img, hsv_padded_img, imgIn, h_tiles,
↪w_tiles, blocks, limit, pixels_per_block, tile_h, tile_w)
return out_img

```

0.19 10. Testiranje implementacije CLAHE algoritma

Na kraju implementacije, radi lakšeg testiranja i upoređivanja rezultata, uvedene su **pomoćne funkcije** kako bi se izbeglo ponavljanje istog koda.

0.19.1 Pomoćne funkcije

Implementirane su tri pomoćne funkcije:

plot_img(img, output) Ova funkcija služi za vizuelni prikaz rezultata: - **Leva slika** – originalna ulazna slika
- **Desna slika** – slika nakon primene CLAHE algoritma

Na ovaj način se jednostavno može vizuelno proceniti efekat algoritma.

execute_dos_clahe(numTiles, clip_limit) Ova funkcija: 1. Učitava test sliku
2. Poziva funkciju `dosCLAHE` sa zadatim parametrima `numTiles` i `clip_limit`
3. Meri vreme izvršavanja algoritma
4. Prikazuje originalnu i obrađenu sliku korišćenjem `plot_img`
5. Ispisuje vreme izvršavanja u sekundama

Cilj ove funkcije je da omogući jednostavno testiranje različitih konfiguracija **broja blokova** i **clip limita** za sopstvenu implementaciju CLAHE algoritma.

execute_adapt_hist(clip_limit) Ova funkcija služi za poređenje sa ugrađenom implementacijom: 1. Učitava istu test sliku
2. Poziva `skimage.exposure.equalize_adapthist`
3. Meri vreme izvršavanja ugrađene funkcije
4. Prikazuje rezultat pomoću `plot_img`
5. Ispisuje vreme izvršavanja

Na ovaj način se omogućava direktno poređenje: - kvaliteta rezultata
- brzine izvršavanja

između moje CLAHE implementacije** i **ugrađene `adapt_hist` funkcije.

```
[ ]: def plot_img(img, output):
    figure, ax = plt.subplots(1,2,figsize=(14,9))
    ax[0].imshow(img)
    ax[1].imshow(output)
    plt.tight_layout()
    plt.show()
```

```
[ ]: def execute_dos_clahe(numTiles, clip_limit, img):
    start=time.time()
    output = dosCLAHE(img, numTiles = numTiles, limit = clip_limit)
    end = time.time()
    print("Moja:")
    print("Execution time:",round(end-start,3),"s")
    plot_img(img, output)
```

```
[ ]: def execute_adapt_hist(clip_limit, img):
    start_adapt = time.time()
    output_adapt = ski.exposure.equalize_adapthist(img, clip_limit = clip_limit)
    end_adapt = time.time()
    print("Ugradjena:")
    print("Execution time:",round(end_adapt - start_adapt,3),"s")
    plot_img(img, output_adapt)
```

0.20 Testiranje i poređenje - realizacija

Algoritam je testiran za različite: - brojeve blokova: (1×1), (4×4), (8×8), (16×16) - vrednosti clip limita: 0, 0.01, 0.1, 1

Rezultati su upoređeni sa ugrađenom funkcijom: `skimage.exposure.equalize_adapthist`.

```
[ ]: # --- TEST RGB---
train_img = ski.io.imread(data_dir + "train.jpg")

# clip limit 0
execute_dos_clahe((1,1), 0, train_img)
execute_dos_clahe((4, 4), 0, train_img)
execute_dos_clahe((8, 8), 0, train_img)
execute_dos_clahe((16,16), 0, train_img)
execute_adapt_hist(0, train_img)

# clip limit 0.01
execute_dos_clahe((1,1), 0.01, train_img)
execute_dos_clahe((4, 4), 0.01, train_img)
execute_dos_clahe((8, 8), 0.01, train_img)
execute_dos_clahe((16,16), 0.01, train_img)
execute_adapt_hist(0.01, train_img)
```

```

# clip limit 0.1
execute_dos_clahe((1,1), 0.1, train_img)
execute_dos_clahe((4, 4), 0.1, train_img)
execute_dos_clahe((8, 8), 0.1, train_img)
execute_dos_clahe((16,16), 0.1, train_img)
execute_adapt_hist(0.1, train_img)

# clip limit 1
execute_dos_clahe((1,1), 1, train_img)
execute_dos_clahe((4, 4), 1, train_img)
execute_dos_clahe((8, 8), 1, train_img)
execute_dos_clahe((16,16), 1, train_img)
execute_adapt_hist(1, train_img)

```

```

[ ]: # --- TEST GRAYSCALE ---
lena_rgb = ski.io.imread("lena.png")
lena_gray = ski.color.rgb2gray(lena_rgb)
lena_gray = (lena_gray * 255).astype(np.uint8)

# clip limit 0
execute_dos_clahe((1,1), 0, lena_gray)
execute_dos_clahe((4, 4), 0, lena_gray)
execute_dos_clahe((8, 8), 0, lena_gray)
execute_dos_clahe((16,16), 0, lena_gray)
execute_adapt_hist(0, lena_gray)

# clip limit 0.01
execute_dos_clahe((1,1), 0.01, lena_gray)
execute_dos_clahe((4, 4), 0.01, lena_gray)
execute_dos_clahe((8, 8), 0.01, lena_gray)
execute_dos_clahe((16,16), 0.01, lena_gray)
execute_adapt_hist(0.01, lena_gray)

# clip limit 0.1
execute_dos_clahe((1,1), 0.1, lena_gray)
execute_dos_clahe((4, 4), 0.1, lena_gray)
execute_dos_clahe((8, 8), 0.1, lena_gray)
execute_dos_clahe((16,16), 0.1, lena_gray)
execute_adapt_hist(0.1, lena_gray)

# clip limit 1
execute_dos_clahe((1,1), 1, lena_gray)
execute_dos_clahe((4, 4), 1, lena_gray)
execute_dos_clahe((8, 8), 1, lena_gray)
execute_dos_clahe((16,16), 1, lena_gray)
execute_adapt_hist(1, lena_gray)

```

0.21 Analiza uticaja parametara na izvršavanje funkcije

U nastavku je analiziran uticaj parametra `limit` na ponašanje algoritma **dosCLAHE**, pri različitom broju blokova po širini i visini slike.

0.21.1 1. `limit = 0`

U slučaju kada je `limit = 0`, bez obzira na broj blokova po širini i visini, **ne dolazi do odsecanja histograma**.

Histogram ostaje neizmenjen, ne postoji višak vrednosti koji bi se redistribuirao, pa transformaciona funkcija postaje identitet.

Kao posledica toga, **izlazna slika je identična ulaznoj slici**, jer se nad njom ne vrši nikakva kontrastna korekcija.

Razlika u odnosu na ugradjenu funkciju (`equalize_adapthist`) potiče od činjenice da ta implementacija **ignoriše slučaj `limit = 0`** (uslov `limit > 0`).

0.21.2 2. `limit = 0.01`

Za malu vrednost limita (0.01) histogram se blago odseca, što dovodi do **poboljšanja lokalnog kontrasta**.

Efekat CLAHE algoritma je primetan, ali nenametljiv — detalji su bolje vidljivi, dok se šum ne pojačava značajno.

Ova vrednost limita je pogodna za slike kod kojih je potrebna **umerena korekcija kontrasta**, bez narušavanja prirodnog izgleda slike.

0.21.3 3. `limit = 0.1`

Povećanjem limita na 0.1 dolazi do **izraženijeg odsecanja histograma**, što rezultuje jačim povećanjem lokalnog kontrasta.

Detalji u tamnim i svetlim regionima slike postaju naglašeniji, ali se u pojedinim slučajevima može primetiti i blago pojačavanje šuma.

Ova vrednost predstavlja kompromis između jačine korekcije i očuvanja kvaliteta slike.

0.21.4 4. `limit = 1`

Kada je `limit = 1`, praktično **ne postoji ograničenje histograma**.

Rezultat je veoma izražen lokalni kontrast, ali uz potencijalno **prekomerno naglašavanje šuma**.

0.22 Vizuelna analiza uticaja broja blokova

0.22.1 (1, 1)

Kod podele slike na samo jedan blok primećuje se poboljšanje kontrasta u tamnim delovima slike. Međutim, ovo poboljšanje dolazi na račun svetlih delova – kontrast u njima je degradiran, zbog čega ti delovi postaju slabije čitljivi.

Ovakvo ponašanje je očekivano, jer se algoritam u ovom slučaju ponaša slično globalnoj histogram equalizaciji.

0.22.2 (4, 4)

Za podelu slike na (4, 4) blokova primećuje se značajno poboljšanje u odnosu na (1, 1) slučaj:

- Svetli delovi (npr. prozor) imaju bolji kontrast
- Tamni delovi (npr. unutrašnjost voza) postaju jasniji

Ipak, pri pažljivijem posmatranju mogu se uočiti diskretnih blokovi(recimo na $x = 200$ vrednosti za `clip_limit = 0.01`).

Iako su rezultati vidno bolji nego kod (1, 1), obrada i dalje nije u potpunosti optimalna.

0.22.3 (8, 8)

Kod podele na (8, 8) blokova kontrast je dodatno poboljšan i lokalni detalji su izraženiji. Međutim, javlja se i neželjeni efekat u vidu šuma:

- Sivi delovi na zidu postaju uočljiviji
- Sitni detalji su prenaplašeni

Ovo ukazuje na to da algoritam počinje da reaguje i na vrlo male lokalne varijacije u intenzitetu.

0.22.4 (16, 16)

Za veliki broj blokova (16, 16) dolazi do preagresivne lokalne obrade:

- Tamni delovi (npr. kauč) postaju znatno tamniji
- Pojavljuje se izražen šum na zidu
- Kontrast između boja postaje prejak

Posebno je uočljivo da je plava boja prenaplašena, dok figura devojke deluje vizuelno neprirodno, gotovo kao da je veštački izdvojena iz slike i nalepljena na pozadinu.

Ovaj efekat jasno pokazuje da prevelik broj blokova može narušiti vizuelni realizam slike.

Povećanjem broja blokova raste lokalnost obrade kontrasta, ali i rizik od šuma i artefakata. U praksi, srednji broj blokova (npr. (4, 4) ili (8, 8)) predstavlja najbolji kompromis između poboljšanja detalja i očuvanja prirodnog izgleda slike.